NEXORA — Complete Project Plan

A single guide covering where the project stands today, what's left on the frontend, the backend we need to build, and the blockchain layer on top of it. Plain English. Read top to bottom.

Last updated: 2026-04-21

Part 1 — Where we are today

NEXORA is a Polymarket-style prediction market — users bet "will X happen?" and get paid if they're right. The project runs on Next.js 16 with a glassmorphism "NEXORA" UI.

What's already built

- Beautiful frontend with ~17 pages (home, markets, dashboard, portfolio, leaderboard, wallet, profile, settings, etc.).
- A custom `server.js` with socket.io for real-time trade updates.
- A simple AMM (Automated Market Maker) formula for pricing.
- An admin page that can create and resolve markets.

What's fake right now

- All market data is hardcoded mock data.
- Wallet balances are fake numbers in a React context.
- No login — users get a random ID on page load.
- Data vanishes every time the server restarts.
- No blockchain, no real money, no security.

In short: the UI is ~75% done, but everything behind it is a placeholder.

Part 2 — What's left on the frontend

Most pages exist. Here's what actually needs fixing before we can call the frontend complete.

Bugs to fix

- **Dashboard settings tabs don't work** — clicking tabs doesn't switch content.
- **Duplicate routes** — `/market/[id]` and `/markets/[id]` both exist; same with `/portfolio` and `/dashboard/portfolio`. Pick one, delete the other.
- **Dead links** — account drawer has "Transactions" and "Analytics" links that go nowhere.

Half-finished pieces

- **Comments component** — visible on market pages but has no backend.
- **Unused layout components** — `Sidebar.tsx`, `TopNav.tsx`, `RightPanel.tsx` are left over; delete or wire them up.
- **Trade modal types** — uses `as any` casts that should be proper TypeScript.

Missing flows

- **Mobile wallet display** — balance shows on desktop but disappears on mobile with no replacement.
- **Dark/light mode toggle** — button exists in settings but doesn't actually toggle.
- **Search filters** — only the Trending page has search; home and dashboard don't.
- **User-facing market creation** — only admins can create markets today.

Polish before launch

- Loading skeletons for when real data is fetching.
- Empty states ("You have no positions yet — here's how to start").
- Error messages + retry for failed trades.
- Toast notifications when a trade confirms.
- Accessibility — keyboard navigation, screen-reader labels, focus traps in modals.

Estimated time to finish frontend: 1 week of focused work, after the backend is wired in.

Part 3 — Backend plan

The backend we need, in order.

Decisions to make first

Four questions drive everything that follows:

1. **Is this real money or play money?** Real money means blockchain, audits, regulatory care. Play money means simpler and faster.
2. **AMM only, or add an order book?** AMM is what we have and is simpler. Order book is what real Polymarket uses today, but takes 3–4x longer to build.
3. **How do users sign in?** Email/password, Google/Twitter, wallet, or all of them?
4. **Heads-up on Next.js 16** — it has breaking changes the AI didn't train on. We read the docs in `node_modules/next/dist/docs/` before writing routes.

Phase 1 — Give the backend a memory (~2 days)

Right now, restarting the server erases every market and trade. We fix that with a real database.

- Use **Postgres** (standard reliable database) with **Prisma** (friendly code that talks to it).
- Tables needed: Users, Markets, Positions, Trades, WalletBalance, Comments, Notifications.
- Move the hardcoded starting markets into a seed script.
- Rewrite the socket.io handlers to save and load from the database.

Phase 2 — Real login (~2 days)

Today, every page refresh gives you a new random identity. We fix that.

- Use **Auth.js v5** (the standard Next.js auth library).
- Support email+password and one OAuth option (Google or Twitter).
- Lock admin actions (create market, resolve market) to admin accounts only.
- New signups get a starting play-money balance (say $1000) so they can try it out.

Phase 3 — Real API (~2 days)

Replace mock endpoints with ones that actually read and write the database.

- List markets (with search, filter by category, sort).
- Get single market with its order book / pool state.
- Place a trade (server validates everything and writes atomically so you can't get weird half-states).
- Get my portfolio, get my activity, get the leaderboard.
- Comments (post, list, reply).
- Rate limiting so nobody can spam us.

Phase 4 — Fix the pricing math (~1 day)

The current formula double-counts. Pick a proper AMM:

- **CPMM (Uniswap-style)** — simple, has built-in slippage. Recommended.
- **LMSR** — what Polymarket originally used. Better when liquidity is thin.

Add a small platform fee (2% is standard). **Put the math in one file with real unit tests** — pricing bugs = lost money.

Phase 5 — Real-time done right (~1 day)

Socket.io works, but right now it broadcasts everything to everyone. We fix that.

- Each market gets its own "room" — you only get updates for markets you're watching.
- Only send what changed (a new trade), not the entire state every time.
- Authenticate socket connections so random people can't inject fake trades.

Phase 6 — Market lifecycle (~2 days)

- **Cron job** — every minute, close any market whose end time has passed.
- **Admin resolution flow** — closed markets appear on admin page, admin picks the winner, payouts run automatically.
- **Notifications** — notify users when their trade fills, market closes, or they win.

Phase 7 — Blockchain (see Part 4 — this is big)

Phase 8 — Deploy and monitor

- **Host Next.js on Vercel.** It's designed for exactly this.
- **Host `server.js` separately** (Railway or Fly.io) because socket.io needs a long-running process. Alternative: swap socket.io for Pusher or Ably and stay fully on Vercel.
- **Database**: Neon or Supabase Postgres.
- **Queues for background jobs**: BullMQ + Redis (Upstash works great on serverless).
- **Error tracking**: Sentry.
- **Logs**: Axiom.
- **Environment variables**: create `.env.example` with every key someone needs to run this locally.

Part 4 — Blockchain & smart contracts

The big upgrade: replace fake money with real money held securely by code.

Why add blockchain?

Right now the server holds everything. Users have to trust us:

- Trust us not to change their balance.
- Trust us not to reverse trades.
- Trust us not to disappear with their money.

Blockchain makes all of that unnecessary. Users hold their own money. A smart contract handles the bets automatically. Nobody — including us — can tamper with it.

What is a smart contract? (Vending machine analogy)

A vending machine is a mini-machine that follows rules: put money in, press a button, get a snack. You don't have to trust the operator because the machine does exactly what it's built to do.

A smart contract is the same, but for digital money. We write the rules once, put it on the blockchain, and from then on it just runs. It can't be stopped, edited, or reversed — so we have to get it right before going live.

Which blockchain platform?

We need one that's cheap (for lots of trades), Ethereum-compatible (so we can reuse existing audited code), and widely used (for wallets and USDC).

Chain	Trade cost	Good for us?
Ethereum mainnet	$1–$20	No — too expensive
Polygon	**$0.001–$0.01**	**Yes — what real Polymarket uses**
Base	$0.01–$0.05	Second choice, Coinbase-backed
Arbitrum / Optimism	$0.01–$0.10	Fine, similar to Base
Solana / Sui / Aptos	cheap	No — different language, full rewrite

We pick Polygon. Same chain as real Polymarket, pennies per trade, same code works on Base / Arbitrum later if we ever want to move.

What language?

Solidity. Standard for every Ethereum-compatible chain. We'll write ~500–800 lines total — most of the heavy lifting is already done by audited code we reuse.

What we build vs. what we reuse

Reuse (already built by others, already audited):

- **USDC** — the dollar-pegged stablecoin. Circle made it; we just point to their contract.
- **Gnosis Conditional Tokens** — turns a bet into YES shares and NO shares. Already on Polygon; we just call it.
- **OpenZeppelin** — safe building blocks (access control, pause switches, reentrancy guards).

Our own code (only three contracts):

1. **AMM** — a mini-machine per market. Holds YES/NO shares, quotes prices, takes a 2% fee.
2. **Factory** — creates a new AMM when an admin makes a new market.
3. **Resolver** — called by the admin multisig to declare the winner.

How "who won?" gets decided

Two options:

- **Admin decides (simple)** — an admin checks the result, clicks resolve. Fast and cheap, but users have to trust the admin.
- **Oracle decides (decentralized)** — a system called **UMA Optimistic Oracle** lets anyone propose an answer, with a time window for anyone to challenge. This is what real Polymarket uses today.

Plan: start with admin resolution, add UMA later. The admin isn't one person — it's a **multisig wallet** where 3 out of 5 trusted people must agree, so no single person can cheat.

Blockchain build steps

B1 — Write and test contracts (1 week)

- Install **Foundry** (the modern tool for writing Solidity).
- Write the AMM, Factory, and Resolver.
- Test every scenario: make a market, buy, sell, resolve, claim winnings.
- Deploy to **Polygon Amoy** (a free test network).

B2 — Connect wallets to the website (3 days)

- Add **RainbowKit** — the nice "Connect Wallet" button.
- Users sign in by connecting MetaMask (or similar).
- Show real USDC balance in the navbar.
- Replace mock trades with real blockchain transactions.

B3 — Mirror the blockchain into our database (3 days)

- Reading the blockchain directly is slow. We run an **indexer** — a small program that watches the chain and copies data into our database.
- Website reads from the database (fast), but the blockchain is the real source of truth.
- Use **Ponder** or **Envio** — they're built exactly for this.

B4 — Resolution flow on chain (2 days)

- Admin UI generates a multisig transaction.
- Once 3 of 5 admins sign, the contract pays out automatically.
- Winners see a "Claim" button to collect their USDC.

B5 — Safety checks (1 week minimum)

- Run **Slither** and **Mythril** (automatic bug scanners).
- Fuzz tests — throw random inputs at the contracts.
- Put a "pause" switch on everything in case we spot an issue.

B6 — Before real money

- **Professional audit** by a security firm. Costs $10k–$30k. **Non-negotiable.**
- **Bug bounty** — pay ethical hackers to find holes first.
- Only then flip to Polygon mainnet.

Part 5 — Full timeline

This is the honest end-to-end estimate, assuming one experienced developer working full-time.

Phase	Time	What you get
Frontend cleanup (bugs, duplicates)	2 days	Polished UI, no more mock clutter
Backend Phase 1 (database)	2 days	Data survives restarts
Backend Phase 2 (auth)	2 days	Real user accounts
Backend Phase 3 (API)	2 days	Real endpoints, no more mocks
Backend Phase 4 (fix AMM math)	1 day	Correct, tested pricing
Backend Phase 5 (secure real-time)	1 day	Safe, efficient socket.io
Backend Phase 6 (market lifecycle)	2 days	Auto-close markets, payouts
Frontend polish (loading, empty, errors, a11y)	3 days	Production-grade UX
Backend Phase 8 (deploy + monitoring)	2 days	Live on internet
MILESTONE Paper-trading launch	**~3 weeks**	**Working demo with fake money**
Blockchain B1 (contracts)	1 week	Contracts on test network
Blockchain B2 (wallet UI)	3 days	Real wallet integration
Blockchain B3 (indexer)	3 days	Fast reads from on-chain data
Blockchain B4 (resolution)	2 days	Admin multisig flow
Blockchain B5 (safety)	1 week	Static analysis, fuzz tests, pause
MILESTONE Testnet launch with real blockchain	**~3 weeks after paper launch**	**Full system on Polygon Amoy**
Blockchain B6 (audit + fixes)	4–6 weeks	Professional review complete
MILESTONE Mainnet deployment	1 week	**Real money. Real users.**

Total: ~2–3 months to real mainnet launch.

Part 6 — Costs

Thing	Cost
Development (testnet, paper)	$0
Database (Neon/Supabase free tier)	$0 to start
Hosting (Vercel free tier + Railway hobby)	~$5–$10/month
Sentry + Axiom (free tiers)	$0
Professional smart-contract audit	**$10k–$30k one-time**
Bug bounty (optional)	$1k–$10k per real issue
Deploying to mainnet (gas)	~$5 one-time
Each user trade (gas, paid by user)	$0.001–$0.01

Important: if we can't afford the audit, we stay on testnet and call it a demo. Unaudited contracts with real money on mainnet is how the news stories start.

Part 7 — Risks and how we handle them

Risk	Our plan
Smart contract bug loses user funds	Tests, static analysis, professional audit, pause switch
Admin goes rogue	3-of-5 multisig — no single admin has power
User loses their wallet / seed phrase	Non-custodial, can't help. Clear warnings at signup
Regulatory changes	Non-custodial model; geo-block countries if required
Polygon network has issues	Contracts are portable — can redeploy on Base or Arbitrum
Server goes down	Blockchain is source of truth, database is just a cache
Mass-spam attacks on comments/trades	Rate limiting + captcha on signup

Part 8 — Decisions we need from you

Before we start, confirm these:

1. **Real money or paper money?** Real means full blockchain + audit ($10k+). Paper means ship in ~3 weeks and skip chain.
2. **AMM only or AMM + order book?** AMM is 1 week, order book is 3–4.

3. **Login options** — email? Google? Twitter? Wallet only? All?
4. **Chain** — Polygon (confirmed) or want to consider Base?
5. **Resolution** — start with admin multisig (recommended) or jump straight to UMA oracle?
6. **Audit budget** — confirm we have $10k–$30k reserved, or accept staying on testnet.

Part 9 — What to do first

Whatever path we pick, the first two weeks are the same:

1. **Clean up frontend bugs** (duplicate routes, broken tabs, dead links) — half a day.
2. **Phase 1: Database.** This unblocks everything. Without it, restarting the server destroys the app.
3. **Phase 2: Auth.** Real logins make everything else real.
4. **Phase 3: Real API.** Replace every mock with a real endpoint.

After that the paths diverge: either finish paper-trading launch (Phases 4–6, 8) or start blockchain (B1) in parallel with the later phases.

End of plan.